Übungsblatt 2: Speicherhierarchie und Caches

Mikrocomputertechnik 3 · Papieraufgaben und optionaler Simulatorvergleich

Ziel und Arbeitsweise

Sie erklären Lokalität, rechnen die mittlere Speicherzugriffszeit aus und verfolgen Cache-Zustände. Alle Pflichtaufgaben sind ohne Rechner und ohne Simulator lösbar.

Bearbeitungszeit: etwa 60–75 Minuten. **Orientierungspunkte:** 32 Punkte. Für die vier Schulstunden sind insgesamt 55 Minuten angeleitete Arbeitsphasen vorgesehen: Aufgabe 1 (10 Minuten), Aufgaben 2–3 (15 Minuten), Aufgaben 4–5 (15 Minuten) und Aufgabe 6 (15 Minuten). Gemeinsame Besprechungen erfolgen in den Vorlesungsphasen; die längere Zeitangabe gilt für selbstständiges Bearbeiten. Begründen Sie Ihre Antworten; zu Rechnungen gehören Formel, Rechenweg und Einheit. Hexadezimale Adressen tragen das Präfix `0x`. Eine Adresse bezeichnet ein Byte.

Wir verwenden ausdrücklich vereinfachte Lehrmodelle: ein Prozessor, normaler cachefähiger Speicher, abgeschlossene Anfragen, kein Vorabladen und keine zusätzlichen Zugriffe. „Kalt“ bedeutet, dass alle Valid-Bits zunächst null sind. Valid ist das Gültigkeitsbit, Dirty das Änderungsbit einer Write-Back-Line. Zähler und Zeiten der Modelle sind keine Messungen eines übersetzten C-Programms.

Aufgabe 1: Lokalität begründen (4 Punkte)

Die beiden Folgen beginnen jeweils unabhängig mit einem kalten Cache und 64-Byte-Lines. Es findet keine Verdrängung statt.

- A: `0x1000, 0x1000, 0x1000`
- B: `0x1000, 0x1004, 0x1008`

1. Ordnen Sie zeitliche und räumliche Lokalität zu. Begründen Sie die Zuordnung. (2 P)
2. Bestimmen Sie für jede Folge Hits und Misses. Erklären Sie, warum eine verstreute verkettete Liste mit demselben Elementumfang schlechtere räumliche Lokalität haben kann, aber nicht muss. (2 P)

Ihr Rechenweg und Ihre Begründung:

Aufgabe 2: AMAT auf zwei Wegen (5 Punkte)

Hit Time $t_H = 1$ Takt, **zusätzliche** Miss Penalty $p = 100$ Takte. AMAT bedeutet Average Memory Access Time, mittlere Speicherzugriffszeit.

1. Eine Folge mit 100 Anfragen hat 95 Hits und 5 Misses. Berechnen Sie die Miss Rate und AMAT. Kontrollieren Sie das Ergebnis durch Aufsummieren aller Einzelzeiten. (3 P)
2. Berechnen Sie AMAT bei 80 % Hit Rate. (1 P)
3. Ist das Verhältnis der beiden AMAT-Werte bereits der Beschleunigungsfaktor eines ganzen Programms? Begründen Sie. (1 P)

Ihr Rechenweg:

Aufgabe 3: Adressfelder und Blockgrenzen (6 Punkte)

Referenzmodell aus der Vorlesung: 32-Bit-Byteadressen, $C = 32$ KiB Datenkapazität, $B = 64$ Byte je Line. Ein KiB hat 1024 Byte. Tags und Zustandsbits zählen nicht zu C.

1. Bestimmen Sie Anzahl der Sets sowie Bitbreiten von Tag, Index und Offset für Direct Mapped und für 4-Way. Erklären Sie, warum der Offset gleich bleibt. (3 P)
2. Zerlegen Sie `0x12345678` für Direct Mapped. Geben Sie Index und Offset dezimal, das Tag hexadezimal an. Kontrollieren Sie durch Rückrechnung. (2 P)
3. Eine andere Anfrage hat die **Dezimaladresse** 1000. Geben Sie bei 64-Byte-Lines Blockbeginn, Blockende und Offset an. (1 P)

Ihre Rechnung:

Aufgabe 4: Zugriffstabelle und Ersetzung (6 Punkte)

4a: Ein Platz oder vier Plätze? (4 Punkte)

Verwenden Sie das Referenzmodell aus Aufgabe 3. Beide Caches beginnen kalt. Alle Anfragen sind Daten-Loads; vier Ways verwenden zunächst freie Plätze.

Schritt	Adresse	DM: Index	DM: Tag	DM: Hit/Miss	4-Way: Hit/Miss
1	`0x1000`				
2	`0x1004`				
3	`0x9000`				
4	`0x1000`				
5	`0x9000`				
6	`0x1004`				

DM steht für Direct Mapped. Füllen Sie die Tabelle aus. Nennen Sie für jeden Cache Hits, Misses und Hit Rate. Begründen Sie, warum die zusätzlichen DM-Misses Konflikt-Misses und keine Kapazitäts-Misses sind.

4b: FIFO oder LRU? (2 Punkte)

Ein getrenntes, zunächst leeres 4-Way-Set erhält die Blockfolge `A, B, C, D, A, E`. Alle Blöcke haben denselben Set-Index.

Geben Sie für FIFO (First In, First Out) und LRU (Least Recently Used) die Reihenfolge „alt nach neu“ nach dem zweiten A sowie den durch E verdrängten Block an. Erklären Sie die Wirkung eines Hits auf beide Regeln.

Ihre Zählung und Begründung:

Aufgabe 5: Schreiben und alte Blockadresse (5 Punkte)

Direct Mapped wie Aufgabe 3, Write-Back und Write-Allocate, keine Puffer. Zunächst ist die Line `0x1000` gültig und sauber (Dirty = 0). Das betrachtete Wort hat im Cache und in der nächsten Ebene den Wert 0. `0x9000` ist nicht im Cache; dort enthält die nächste Ebene den Wert 9.

Verfolgen Sie: `Store 5 nach 0x1000; Load 0x1000; Store 7 nach 0x1000; Load 0x9000`. Nur diese vier Anfragen werden gezählt.

1. Füllen Sie Zustand und eventuelles Writeback nach jedem Schritt aus. (3 P)

Schritt	Cache-Blockbasis	Wortwert	Dirty	Wert unten bei `0x1000`	Writeback-Adresse
Store 5					
Load					
Store 7					
Load `0x9000`					

2. Warum muss der letzte Schritt gegebenenfalls an die alte Blockadresse zurückschreiben? (1 P)

3. Was wäre bei einem Store-Miss mit No-Write-Allocate anders? Ist diese Regel durch Write-Back schon festgelegt? (1 P)

Ihre Erläuterung:

Aufgabe 6: C-Layout als Papieraufgabe (6 Punkte)

Es gilt `uint32_t a[16][16]`: 16 Zeilen, 16 Spalten, 4 Byte pro Element, Basis `0x1000`. Die Elemente sind initialisiert. Betrachten Sie zwei **definierte Modellspuren**, je genau ein Daten-Load pro Element. Andere Zugriffe und Compileroptimierungen gehören nicht zur Zählung.

```
/* A: zeilenweise; sum ist ein uint64_t-Akkumulator mit Startwert 0. */
for (unsigned r = 0; r < 16; ++r)
    for (unsigned c = 0; c < 16; ++c)
        sum += a[r][c];

/* B: unabhängig davon spaltenweise, sum zuvor wieder 0. */
for (unsigned c = 0; c < 16; ++c)
    for (unsigned r = 0; r < 16; ++r)
        sum += a[r][c];
```

`uint32_t` und `uint64_t` sind vorzeichenlose Ganzzahltypen mit 32 beziehungsweise 64 Bit; `r` und `c` sind Zeilen- und Spaltenindex.

1. Leiten Sie die Elementadresse her und geben Sie die ersten vier Adressen beider Spuren an. (2 P)
2. Kleines Modell: 512 Byte Datenkapazität, Direct Mapped, 64-Byte-Lines, vor jeder Variante kalt. Leiten Sie den Set-Index der Zeile r her und zählen Sie Hits und Misses für beide vollständigen Spuren. (3 P)
3. Ändern Sie nur die Datenkapazität auf 1024 Byte. Was ändert sich an der spaltenweisen Bilanz? Begründen Sie. (1 P)

Ihr Rechenweg:

Optional: Simulatorvergleich (ohne Punkte)

Wenn Ripes, ein visueller RISC-V-Simulator, verfügbar ist, vergleichen Sie eine explizite Load-Spur mit der Papierlösung. Dokumentieren Sie Version, Cache-Größe, Blockgröße, Ways, Anfangszustand und welche Datenanfragen tatsächlich gezählt werden. Setzen Sie den Cache vor jedem Vergleich zurück.

Behandeln Sie Trefferzahlen als Analyse der Spur. Das Cache-Modell von Ripes ist kein vollständiges Cache-Latenzmodell und belegt keinen allgemeinen Geschwindigkeitsfaktor. Zusätzliche Datenzugriffe eines Simulatorprogramms müssen gesondert berücksichtigt werden.